AccountingAgents: A Multi-Agent LLM Framework for Accounting Firm Automation

Paul Desrochers
Independent Researcher · Quebec, Canada · pdesrochers.ai.research@gmail.com
Preprint · April 2026

Abstract
We present AccountingAgents, a novel multi-agent Large Language Model (LLM) framework inspired by the organizational structure of professional accounting firms. Drawing a direct parallel to TradingAgents [1] in financial trading, AccountingAgents defines specialized agent roles — Ingestion, Reconciliation, Reporting, Compliance, Accounts Receivable (AR), Accounts Payable (AP), and Onboarding — coordinated by a central Supervisor via a structured SharedState. Unlike existing commercial solutions (Intuit Assist, Pilot AI) that operate as closed, monolithic systems, our framework is open, composable, and MCP-native, leveraging official Model Context Protocol servers (e.g., QuickBooks Online MCP, Google Workspace MCP) as representative integrations. A key contribution is the design of an asynchronous Human-in-the-Loop (HITL) mechanism that suspends agent threads via LangGraph's interrupt() primitive, notifies the supervising accountant by any instant messaging mechanism (eg Gmail, Slack, etc), and resumes execution upon webhook-delivered approval — enabling mobile, real-world oversight without a dedicated console. The framework targets small-to-medium accounting firms underserved by enterprise solutions.
Keywords: multi-agent systems · LLM · accounting automation · human-in-the-loop · MCP · LangGraph · QuickBooks Online

1. Introduction

Autonomous agents powered by Large Language Models (LLMs) have demonstrated transformative potential across domains requiring complex, multi-step reasoning and tool use. In finance, the TradingAgents framework [1] established a compelling paradigm: modelling the multi-agent decision process of a professional trading firm, with specialized roles for analysts, researchers, traders, and risk managers communicating through structured state rather than unstructured natural language. This design addresses two fundamental limitations of prior systems — the absence of realistic organizational modelling, and the "telephone effect" of context corruption across long natural language histories.

We observe that the accounting profession presents an analogous opportunity. A typical accounting firm processes hundreds of documents weekly — invoices, bank statements, tax filings, client correspondence — executing workflows that are highly structured, role-based, and subject to strict regulatory constraints. Despite a booming market for AI-assisted accounting (projected at $96.69B by 2033 [2]), existing solutions are either monolithic commercial products with vendor lock-in, or isolated single-agent tools that address narrow tasks without organizational coherence. The breadth of AI technologies applicable to Finance and Accounting — including multi-agent systems, reinforcement learning, graph neural networks, and neuro-symbolic AI — has been extensively surveyed by Ramachandran [11], confirming both the maturity of the underlying technologies and the scale of the automation opportunity. AccountingAgents addresses this opportunity through a concrete, opinionated framework targeting the specific organizational structure and operational constraints of small-to-medium accounting firms.

Beyond automation capability, deployment in regulated environments demands that every agent action be traceable and attributable. Barrak [12] demonstrates empirically that structured handoffs between role-specialized agents prevent silent error propagation and improve system reliability — a finding directly applicable to accounting workflows where errors carry fiduciary and legal consequences. AccountingAgents addresses this requirement by design, through a SharedState architecture that makes every agent delta auditable and a HITL escalation model that ensures human authorization of high-stakes actions.

This paper introduces **AccountingAgents**, a multi-agent LLM framework that replicates the collaborative structure of a professional accounting firm. Our primary contributions are:

- A role-specialized multi-agent architecture mirroring real accounting firm workflows built on LangGraph StateGraph with a structured SharedState;
- Native integration with official MCP servers (QuickBooks Online, Gmail, Google Drive, Google Calendar) eliminating fragile custom API wrappers;
- An asynchronous Human-in-the-Loop (HITL) mechanism using LangGraph interrupt(), any MCP-compatible messaging channel, and a lightweight webhook endpoint enabling mobile approval without a dedicated console;

- A four-level HITL escalation model calibrated to accounting risk profiles (automated, notify, approve, transfer);
- Open-source positioning serving small-to-medium firms underserved by enterprise solutions.

2. Related Work

2.1 Multi-Agent LLM Frameworks in Finance

TradingAgents [1] proposes a stock trading framework with five specialized teams — Analysts, Researchers (Bull/Bear debaters), Traders, Risk Managers, and a Fund Manager — communicating via structured outputs. This design yields measurable improvements in Sharpe ratio and maximum drawdown over single-agent baselines. FinDebate [3] extends debate-driven reasoning to financial analysis reports. A recent arXiv benchmark [4] evaluates four multi-agent orchestration architectures (sequential pipeline, parallel fan-out, hierarchical supervisor-worker, reflexive self-correcting) on 10,000 SEC filings, finding hierarchical architectures occupy the most favorable cost-accuracy Pareto frontier (F1 0.921 at 1.4x cost). AccountingAgents adopts a hierarchical supervisor-worker architecture, consistent with these findings.

2.2 AI in Accounting

Commercial solutions such as Intuit Assist [5], Pilot AI [6], Docyt, and FloQast provide AI-assisted bookkeeping within closed ecosystems. Intuit reports that its AI reduces reconciliation time by ~3x and that 74% of customers post transactions faster [5]. However, these systems are opaque, expensive for small firms, and tightly coupled to the vendor's own regulatory interpretations — leaving no room for jurisdiction-specific customization or third-party extensibility. On the open-source side, panaversity/accounts_ai [7] proposes an AI accounting co-pilot but remains at the architectural design stage without experimental validation. To our knowledge, no published academic work proposes a complete multi-agent LLM framework modelled on accounting firm organizational structure.

2.3 Human-in-the-Loop in Agentic Systems

HITL mechanisms in agentic systems have been studied in the context of code generation (AlphaGPT [8]) and general task automation. LangGraph's interrupt() primitive [9] provides a first-class HITL abstraction: threads are suspended, state is persisted via a checkpointer, and execution resumes upon external signal. Our contribution extends this to an asynchronous, mobile-friendly approval loop via any near-real-time communication mechanism (e.g., email, instant messaging), addressing the practical constraint that accounting supervisors are rarely at a workstation.

2.4 Traceability and Accountability in Multi-Agent Systems

Traceability and accountability are foundational requirements in regulated environments such as accounting, where every financial action must be auditable and attributable. Barrak [12] empirically demonstrates that structured, accountable handoffs between role-specialized agents markedly improve accuracy and prevent the silent propagation of errors common in naive pipelines. Their Planner → Executor → Critic architecture shows that saving records of who did what at each step is not merely a governance concern but a direct driver of system reliability. AccountingAgents inherits this principle through two complementary mechanisms: (1) the SharedState architecture, where each agent writes only its own delta, creating an implicit, auditable trail of every decision and transformation; and (2) the four-level HITL escalation model (N1–N4), which ensures that high-stakes actions — vendor payments, AR escalations, reconciliation corrections — are explicitly authorized before execution and logged upon completion.

3. AccountingAgents Framework

3.1 Design Principles

AccountingAgents is governed by three principles inherited from TradingAgents and adapted to the accounting domain:

- Structured state as the sole communication channel. All agents read from and write to a typed SharedState (Python TypedDict), eliminating context corruption across long conversation histories.
- Delta-based updates. Each agent node returns only the fields it modifies; LangGraph merges these automatically into the global state.
- Framework complexity must be justified. A framework is adopted only when the complexity it manages exceeds the complexity it introduces.

3.2 Overall Framework Organization

The framework comprises seven specialized agents, organized sequentially with conditional routing by the Supervisor. Figure 1 illustrates the overall architecture.

Fig. 1: AccountingAgents Overall Framework Organization

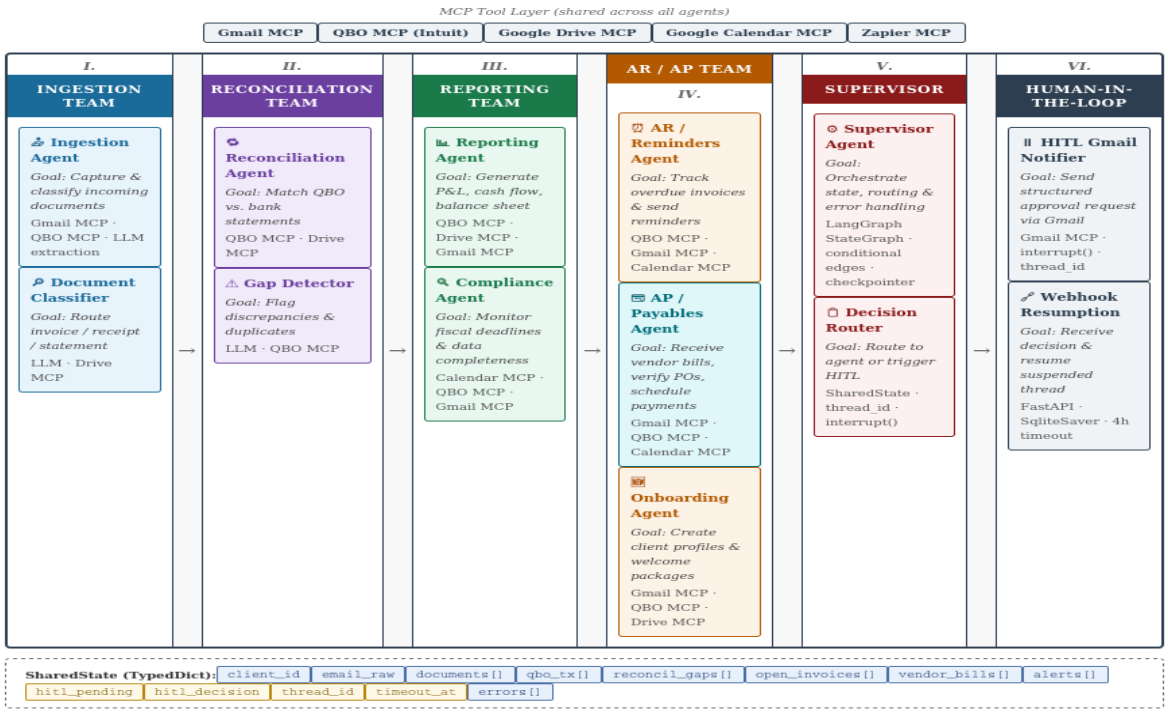

Fig. 1: AccountingAgents Overall Framework Organization. I. **Ingestion Team**: agents capture and classify incoming financial documents via Gmail. II. **Reconciliation Team**: agents match transactions against bank statements and flag gaps. III. **Reporting Team**: agents generate financial statements and monitor compliance deadlines. IV. **AR/AP/Client Team**: agents manage overdue invoices (AR), vendor bills and payment approvals (AP), and client onboarding. V. **Supervisor**: orchestrates shared state, conditional routing, and error recovery via LangGraph. VI. **Human-in-the-Loop (HITL)**: asynchronous approval cycle — interrupt(), Gmail notification, webhook resumption, 4h timeout. All agents communicate exclusively through a structured SharedState (TypedDict), eliminating the "telephone effect" of natural language message passing.

3.3 Agent Roles

Team	Agent	Goal	MCP Tools for the MVP
I. Ingestion	Ingestion Agent Document Classifier	Capture & classify incoming financial documents	Gmail MCP · QBO MCP · LLM
II. Reconciliation	Reconciliation Agent Gap Detector	Match transactions vs. bank statements; flag discrepancies	QBO MCP · Drive MCP
III. Reporting	Reporting Agent Compliance Agent	Generate P&L, cash flow; monitor fiscal deadlines	QBO MCP · Gmail MCP · Calendar MCP
IV. AR / AP / Client	AR Agent AP / Payables Agent Onboarding Agent	Track overdue invoices; approve vendor bills & payments; create client profiles	QBO MCP · Gmail MCP · Calendar MCP
V. Supervisor	Supervisor Decision Router	Orchestrate state, routing, error handling	LangGraph StateGraph · checkpointer
VI. HITL	HITL Notifier Webhook Resumption	Async approval via Gmail; resume suspended thread	Gmail MCP · webhook endpoint · SqliteSaver

Table 1: AccountingAgents role definitions.

3.4 SharedState Design

All inter-agent communication flows exclusively through a structured **SharedState** TypedDict. This design eliminates the "telephone effect" identified in TradingAgents [1] — the progressive corruption of context that occurs when agents communicate through natural language message histories. The SharedState fields are partitioned into two categories:

- Business fields cover document ingestion, transaction data, reconciliation gaps, AR/AP state, reporting artifacts, routing signals, and error logs.
- HITL fields (hitl_pending, hitl_decision, thread_id, timeout_at) manage the suspension and resumption lifecycle.

Each agent node returns only a delta — the subset of fields it modifies — which LangGraph automatically merges into the global state. This ensures agents remain decoupled and that state evolution is auditable.

4. Asynchronous Human-in-the-Loop

4.1 Motivation

Accounting operations carry fiduciary and legal obligations that preclude fully autonomous execution of high-stakes actions — client escalations, reconciliation corrections, vendor payment approvals, new client onboarding — without human oversight. However, a HITL mechanism requiring a supervising accountant to monitor a console is impractical: accounting professionals are frequently in client meetings, on-site visits, or otherwise away from a workstation.

4.2 Four-Level Escalation Model

We define four escalation levels calibrated to accounting risk profiles:

Level	Mode	Trigger	Example
N1	Automatic	Routine, low-risk action	First AR reminder, standard report, small vendor bill < $500
N2	Notify only	Action taken; cancellable within window	Invoice created, client profile drafted, recurring AP payment
N3	Approve (HITL)	High-value or irreversible action	AR escalation >$5k, reconciliation gap, vendor payment >$2k
N4	Transfer	Outside known rules	Client dispute, regulatory anomaly, unrecognized vendor

Table 2: Four-level HITL escalation model.

4.3 Asynchronous HITL Cycle

The N3 HITL cycle proceeds as follows:

- Agent detects an exception condition (e.g., reconciliation gap > threshold, AR escalation required, vendor payment > approval threshold).
- Supervisor calls interrupt(), persisting the complete thread state via a pluggable checkpointer backend.
- The messaging MCP server sends a structured notification (e.g., email, instant message) to the supervising accountant, including client context, agent suggestion, and three action links (Approve / Modify / Block).
- The accountant responds with a single click from any device (iPhone, desktop).
- The click triggers a webhook callback to a lightweight API endpoint, delivering the thread identifier and decision payload.
- LangGraph resumes the suspended thread at the exact interruption point, injecting the decision into SharedState.
- If no response is received within 4 hours, a timeout handler injects status: "timeout" and the Supervisor escalates automatically.

This design requires zero additional tooling beyond components already in the stack: the messaging MCP server, the checkpointer backend, and a minimal webhook endpoint.

5. MCP Tool Integration

AccountingAgents is MCP-native by design. Rather than implementing custom API wrappers for each external service — an approach that creates fragile, maintenance-heavy integrations — all tool access is mediated through official Model Context Protocol servers. This provides two key benefits: (1) authentication and authorization are handled by established OAuth2 flows, and (2) tool availability and capability evolve with the MCP specification independently of the agent framework.

The five MCP servers in the MVP stack are: **Gmail MCP** (gmailmcp.googleapis.com) for all email I/O including HITL notifications; **QBO MCP** (intuit/quickbooks-online-mcp-server [10]) providing CRUD operations on Account, Bill, Customer, Invoice, Vendor, and 7 additional entities; **Google Drive MCP** for document storage; **Google Calendar MCP** for fiscal deadline monitoring; and **Zapier MCP** as a general-purpose bridge to services lacking native MCP servers.

Additional MCP servers can be substituted or added without modifying the agent logic.

6. Limitations and Future Work

The present work is a position paper describing framework design. Several limitations apply. First, we present no experimental results; a companion implementation paper will evaluate agent accuracy, HITL approval rates, and time savings against a baseline of manual processing. Second, the four-level escalation thresholds are defined heuristically; future work should calibrate these empirically per firm profile. Third, the framework currently targets QBO; extension to Xero, Sage, and Microsoft Dynamics 365 would broaden applicability. Fourth, AccountingAgents acts as a jurisdiction-agnostic orchestration layer — regulatory and fiscal specifics remain the responsibility of the underlying accounting software (QBO, Xero, Sage). Extension to additional MCP-compatible accounting backends would broaden applicability across jurisdictions without modifying agent logic.

7. Conclusion

We have presented AccountingAgents, a multi-agent LLM framework modelled on the organizational structure of professional accounting firms. The framework addresses the two core limitations identified in prior work — absence of realistic organizational modelling and inefficient communication interfaces — by adopting the structured SharedState design pioneered by TradingAgents and extending it to the accounting domain. The asynchronous HITL mechanism represents a practical contribution to the deployment of agentic systems in regulated, high-stakes environments where human oversight is legally required but operators are mobile. We invite the research community to build on this framework and to contribute experimental validation.

References

[1] Xiao, Y., Sun, E., Luo, D., & Wang, W. (2025). TradingAgents: Multi-Agents LLM Financial Trading Framework. arXiv:2412.20138.

[2] Grand View Research (2024). AI in Accounting Market Size & Growth Report 2033.

[3] FinDebate: Multi-Agent Collaborative Intelligence for Financial Analysis. arXiv:2509.17395 (2025).

[4] Kulkarni, S. & Kulkarni, Y. (2026). Benchmarking Multi-Agent LLM Architectures for Financial Document Processing. arXiv:2603.22651.

[5] Intuit QuickBooks. AI Agents for Intelligent Business Automation. intuit.com/enterprise/ai-agents (2025).

[6] Pilot AI. World's First Fully Autonomous AI Accountant for SMBs. Press release, February 2026. https://www.globenewswire.com/news-release/2026/02/04/3232133/0/en/Pilot-Unveils-AI-Accountant-A-Major-Leap-Toward-Artificial-General-Intelligence-in-Accounting.html

[7] panaversity. accounts_ai: AI-based accounting co-pilot. github.com/panaversity/accounts_ai (2024).

[8] Wang, X. et al. (2023). AlphaGPT: Human-in-the-Loop Perspective on Research-Driven Alpha Mining for Quantitative Investments.

[9] LangChain Inc. (2024). LangGraph: Agent Orchestration Framework. langchain.com/langgraph.

[10] Intuit. quickbooks-online-mcp-server. github.com/intuit/quickbooks-online-mcp-server (2025).

[11] Ramachandran, A. (2025). Enterprise Finance & Accounting: Automation through Agentic and Multi-Agent AI Systems Leveraging Reasoning Models, Multimodal Interfaces, Reinforcement Learning, Graph Neural Networks, and Neuro-symbolic AI. ResearchGate. researchgate.net/publication/390209544

[12] Barrak, A. (2025). Traceability and Accountability in Role-Specialized Multi-Agent LLM Pipelines. arXiv:2510.07614.